

EBMS

Ethiopian Business Management System

Technical Capabilities & Module Reference Guide

Version 2.2 · September 2026

Platform: FastAPI + PostgreSQL 16 + Docker Compose
Modules: 33 fully integrated business modules
Deployment: On-premises Docker · Nginx TLS · Tailscale VPN
Compliance: ERCA VAT / withholding forms · Amharic UI · Ethiopian calendar
Security: OWASP Top-10 hardened · SIEM built-in

Table of Contents

1.	System Overview	3
2.	Architecture & Technology Stack	4
3.	Authentication & User Management	5
4.	VAT Portal Module	6
5.	Journal Entry & Chart of Accounts	7
6.	Income & Expense Module	8
7.	Transaction Management	9
8.	Ethiopian Payroll Module	10
9.	Inventory Management	11
10.	CPO (Cash Payment Order) Module	12
11.	Bid Tracker Module	13
12.	Letter & E-Signature Module	14
13.	SIEM Security Module	15
14.	Backup & Archive System	16
15.	Version Control Module	16
16.	Multi-Company Portal	17
17.	Learning Management System (LMS)	18
18.	Machinery & Equipment Management	19
19.	REST API v1 & v2	20
20.	Subscription Tiers & Licensing	21
21.	Security & Compliance	22
22.	Deployment & Infrastructure	23
23.	Database Schema Reference	24
24.	Operations Suite Modules	26
25.	Ethiopian-Native, Open Platform & Workflow (v2.2)	27

◆ 1. System Overview

The **Ethiopian Business Management System (EBMS)** is a full-stack enterprise resource planning platform purpose-built for Ethiopian businesses. It integrates twenty-three business modules — spanning financial accounting, tax compliance, HR & payroll, inventory, bid management, projects, procurement, events, e-signatures, and security monitoring — into a single web application accessible from any browser.

Platform	FastAPI (Python 3.12) · Jinja2 templates · PostgreSQL 16
Hosting	Self-hosted Docker Compose (6 services) · Nginx TLS reverse proxy
Remote access	Tailscale VPN (WireGuard) — server not exposed to the internet
Multi-tenancy	Per-company data isolation via PostgreSQL Row-Level Security
Authentication	Session cookies + Bearer token API auth · bcrypt passwords
Tax Compliance	Ethiopian Revenue & Customs Authority (ERCA) aligned VAT rules
Languages	English UI — fully localizable
Modules	23 integrated modules (see sections 3–18 and 24)
API	REST API v1/v2 under /api/ — JSON, CSV, XLSX export
Security	OWASP Top-10 hardened · CSRF protection · SIEM built-in
Subscription model	Starter / Professional / Enterprise tiers
LMS	Integrated learning with HR certification tracking
Equipment	Construction machinery & asset lifecycle management

◆ 2. Architecture & Technology Stack

Backend Framework

Built on **FastAPI** (ASGI) served by **Uvicorn** behind Nginx. Each business module is an independent *APIRouter* registered on startup via the `_reg()` factory, ensuring clean separation and safe fallback if a module fails to load. Starlette middleware layers handle authentication, CSRF, session management, structured logging, GZip compression, rate-limiting and audit-trail in a defined stack order.

Data Layer

Primary storage is **PostgreSQL 16** (containerised, data on a named Docker volume). The full schema is created idempotently at startup from `aws-deployment/init_db.sql`, with incremental changes applied by Alembic migrations. Row-level security policies enforce per-tenant data isolation at the database level — even if an application WHERE clause is accidentally omitted. An async connection pool (*asyncpg*) handles high-concurrency reads. Letter / E-Signature module uses JSON file storage for lightweight, portable document management.

Web server	Nginx 1.27	TLS termination (Let's Encrypt), reverse proxy
App server	Uvicorn (ASGI)	Async HTTP server, 2 workers per service
Framework	FastAPI + Starlette	Routing, middleware, dependency injection
Templates	Jinja2	Server-side HTML rendering
Database	PostgreSQL 16 (Docker)	Primary persistent storage with RLS
Cache	Redis 7 (Docker)	Session store, event bus, rate-limit counters
File storage	Docker volume (+S3 opt.)	Backup archives, DOCX exports, bid documents
Background	APScheduler + host cron	In-app scheduler; nightly DB + data backup via cron
Monitoring	SIEM + structured JSON logs	Security events, audit trail, Prometheus /metrics
Infra	Docker Compose	6 services: web, api, event-worker, nginx, redis, postgres

Middleware Stack (innermost → outermost)

- GZip compression — compresses all responses ≥ 1 KB
- Session middleware — Redis-backed (cookie fallback)
- Rate limiter — SlowAPI enforces per-endpoint limits
- Audit trail — logs every successful POST/PUT/PATCH/DELETE to SIEM
- CSRF auto-inject — hidden token injected into every HTML form
- CSRF validation — validates X-CSRFToken on AJAX/HTMX mutations
- Module license check — blocks unlicensed module access per tenant
- Company context — sets `request.state.company_id` for multi-tenancy
- Auth gate — validates session or Bearer token on every request
- Structured logger — emits one JSON log line per HTTP request
- Request-ID tagger — X-Request-ID header on every response

◆ 3. Authentication & User Management

Authentication System

Secure multi-tenant login system with role-based access control, API token generation, and company-context switching.

- Username/password login with bcrypt hashing (cost factor 12)
- Role-based access: super_admin → admin → manager → user
- Per-company multi-tenancy — users belong to one or more companies
- Bearer token generation for API / programmatic access
- Server-side idle session expiry — 30 minutes by default (SESSION_IDLE_MAX_MINUTES)
- Access-denied page with role requirement display
- Company registration and company selection portal
- CSRF-protected login form with rate-limiting
- Session stored in Redis (fallback to signed cookie)
- Audit log of every login / logout event in SIEM

Technical notes: Passwords hashed with bcrypt. Sessions use server-side Redis store with SameSite=Lax cookies. Bearer tokens are SHA-256 hashed in DB.

/auth/login	GET/POST	Login form and authentication
/auth/logout	GET	Invalidate session and redirect to login
/auth/register	GET/POST	New user self-registration
/auth/portal	GET	User dashboard / module selector
/auth/access-denied	GET	Permission-denied landing page
/auth/users	GET	Admin: list all users (admin+)
/auth/api-tokens	GET	Manage personal API tokens
/company/login	GET/POST	Company-specific login
/company/register	GET/POST	Register new company
/company/select	GET/POST	Switch between companies

◆ 4. VAT Portal Module

VAT Portal

Comprehensive VAT management module aligned with Ethiopian Revenue & Customs Authority (ERCA) requirements. Handles income, expenses, and capital transactions with automatic VAT calculation.

- Income recording with VAT types: standard (15%), exempt, zero-rated
- Expense recording with supplier TIN, receipt number, and VAT breakdown
- Capital investment tracking with investor details
- Automatic VAT calculation and running VAT liability balance
- Financial summary report: total income, expenses, capital, net position
- VAT compliance report exportable to Excel
- Dashboard with period-based filtering (monthly, quarterly, annual)
- Withholding Tax (WHT) support
- Gross / Net / VAT split on every record
- Created-by tracking for audit trail

Technical notes: VATType enum supports: standard_15, exempt, zero_rated, reverse_charge, withholding_2, withholding_15, withholding_30. IncomeCategory and ExpenseCategory enums map to ERCA schedule categories.

/vat/	GET	VAT dashboard with totals
/vat/income	GET	Income list with filters
/vat/income/add	GET/POST	Add new income record
/vat/income/	GET	Income detail view
/vat/expenses	GET	Expense list
/vat/expenses/add	GET/POST	Add new expense
/vat/capital	GET	Capital transactions list
/vat/capital/add	GET/POST	Add capital investment
/vat/financial-summary	GET	Full financial summary report

◆ 5. Journal Entry & Chart of Accounts

Journal Entry System

Double-entry bookkeeping engine with a structured chart of accounts supporting multi-level parent/child account hierarchies.

- Double-entry journal entries — every debit must equal credit
- Multi-line journal entries with any number of debit/credit lines
- Account code auto-completion and validation
- Journal entry status: draft / posted / reversed
- Reference number tracking for external document linkage
- Trial balance generation — aggregates all posted entries
- General ledger view filterable by account, date, and period
- Reversal entries — automatically create offsetting entries
- Excel/CSV export of journal entries and trial balance
- Created-by audit trail on every entry

Technical notes: Uses the GeneralLedger core class. JournalEntryBuilder pattern constructs validated entry objects before persistence.

Chart of Accounts

Structured account hierarchy following Ethiopian accounting standards, supporting assets, liabilities, equity, income, and expense account types.

- Asset / Liability / Equity / Income / Expense account types
- Parent-child account hierarchy (sub-accounts)
- Normal balance tracking (debit vs credit)
- Current balance maintained in real-time
- Import accounts from Excel template
- 25+ pre-seeded standard Ethiopian chart of accounts
- Account activation / deactivation
- Account code uniqueness enforced per company

Technical notes: AccountType and AccountSubType enums defined in models/account.py. Per-company isolation via company_id on every account record.

◆ 6. Income & Expense Module

Income & Expense

Simplified income and expense tracking separate from the full VAT portal, designed for day-to-day business operations with automatic tax calculations.

- Income recording: client name, TIN, category, gross/net/tax split
- Expense recording: supplier, category, deductibility flag, tax breakdown
- Real-time profit/loss calculation on dashboard
- 15% VAT standard rate auto-applied with override option
- Monthly salary data automatically pulled from payroll module
- One-click creation of standard monthly IT expenses (Internet, Software, Support)
- Excel import/export (openpyxl) for bulk data entry
- Session-persistent data — restored across browser sessions
- Category breakdown charts on dashboard
- Filterable by date range, category, and payment method

Technical notes: income_expense_data_store feeds data to both the HTML dashboard and the /api/v1/export/vat_income endpoint.

◆ 7. Transaction Management

Transaction System

Bank statement and ledger transaction import, review, and flagging system for identifying suspicious or out-of-policy transactions.

- Bulk Excel/CSV import of bank transactions
- Import history log with row count, error count, and batch ID
- Transaction flagging: manual flag or auto-flag by rules
- Flagged accounts registry — automatically marks transactions from known accounts
- Individual-name detection: flags transactions with personal names in description
- Review workflow: pending → reviewed with reviewer notes
- Counterparty, account code, reference number fields
- Debit/credit/balance columns with currency support
- Filter by flag status, review status, date range, and account
- Export filtered transactions to CSV/XLSX
- Per-batch undo: delete all records from an import batch

Technical notes: Bulk import uses pandas for parsing. FlaggedAccounts table persisted in PostgreSQL. Auto-flag runs on import, not retroactively.

◆ 8. Ethiopian Payroll Module

Ethiopian Payroll

Full Ethiopian payroll calculation engine compliant with the Ethiopian Income Tax Proclamation, supporting all statutory deductions and employer contributions.

- Ethiopian progressive income tax (PAYE) — 7 tax brackets per proclamation
- Pension fund: employee 7% + employer 11% contributions
- Overtime calculation: regular overtime, holiday, and weekend rates
- Allowances: transport, housing, and other taxable/non-taxable allowances
- Net pay and gross pay calculation with full deduction breakdown
- Employee master data: name, ID, department, position, bank account, TIN
- Date of birth and age tracking
- Manager field for org-chart hierarchy
- Phone number contact field
- Pension number tracking
- Work days per month and hours per day configuration
- CPO (Cash Payment Order) integration — deducts returned amounts from payroll
- Org chart — visual hierarchy tree built from manager field
- Payslip generation — formatted PDF payslip per employee
- Payroll reports: summary by department, cost centre analysis
- Bulk employee import from Excel
- Tax calculator tool — ad-hoc tax estimation without creating payroll
- Ethiopian calendar support

Technical notes: EthiopianPayrollCalculator in models/ethiopian_payroll.py implements the full PAYE algorithm. Employee model includes date_of_birth, phone_number, manager fields added 2026-03-15.

◆ 9. Inventory Management

Inventory Management

Full lifecycle inventory management supporting physical goods, rentable assets, and consumable items with movement tracking.

- Item master: SKU, barcode, serial number, batch number
- Item categories with parent-child hierarchy
- Unit of measure, unit price, and cost price
- Current stock level with minimum stock and reorder point alerts
- FIFO / LIFO / Weighted Average valuation methods
- Inventory movements: purchase, sale, transfer, adjustment, return
- Location-based tracking (from / to location)
- Allocation management — assign items to events/projects
- Return tracking — expected vs actual return dates
- Maintenance scheduling and cost tracking per item
- Requisition management — request items with priority and approval workflow
- Excel/CSV bulk import with import history
- Low-stock dashboard alerts
- Rentable asset flag for event equipment

Technical notes: ValuationMethod enum: FIFO, LIFO, WEIGHTED_AVG. Movements and allocations stored in separate PostgreSQL tables for full history without overwriting balances.

◆ 10. CPO (Cash Payment Order) Module

CPO Module

Tracks Cash Payment Orders issued to vendors and employees, including return deductions that reduce net outstanding balances.

- CPO record: name, date, amount, bid reference, return status
- Bulk Excel import with import history and batch tracking
- Return (deduction) tracking: mark CPOs as returned with date
- Summary dashboard: total issued, returned (deducted), net outstanding
- Record count by status: returned vs outstanding
- Per-company isolation — CPOs belong to specific company context
- SIEM audit logging on every import action
- Export all CPO records to CSV/XLSX via API export endpoint
- List view with filtering and sorting

Technical notes: CPODataStore stores records in PostgreSQL cpo_records table. get_summary() computes returned_amount (sum of returned CPOs) and net_amount (total issued minus returned) in a single query pass.

◆ 11. Bid Tracker Module

Bid Tracker

End-to-end tender and bid management system for tracking open, submitted, won, and lost bids with automated deadline reminders.

- Bid record: title, reference number, organization, category, status
- Status workflow: open → submitted → won / lost / cancelled
- Deadline tracking with configurable reminder days-before flag
- Automated email/notification reminder (reminder_sent flag)
- Bid amount and currency tracking
- Case handler assignment: name and email
- Document attachment: upload multiple files per bid
- Document type classification (technical, financial, legal)
- File size and upload timestamp tracking
- Per-bid activity notes
- Dashboard with status distribution and upcoming deadlines

Technical notes: bid_documents_meta stores file metadata; actual files stored on local FS under web/data/bids/documents/ (or S3 in production). reminder_days_before default is 3 days.

◆ 12. Letter & E-Signature Module

Letters & E-Signatures

Sequential letter management with digital signatures from three designated organizational signatories (Project Manager, Finance Manager, Managing Director). Letters are tracked from composition through all signatures to dispatch.

- Sequential auto-numbered reference IDs (REF-0001, REF-0002, ...)
- Letter composition: subject, recipient, full body text, category
- Three-tier signature workflow: PM → FM → MD
- Canvas-based e-signature capture — draw signatures directly in browser
- Saved signature pads per role — sign once, reuse across letters
- Signature timestamp: exact date and time of each signature captured
- Letter status lifecycle: draft → signed → sent
- DOCX generation using company Templates.docx (or clean A4 fallback)
- Embedded base64 signature images in generated DOCX
- Mail tracker: full event log with timestamp for every action
- Signature preview panel on dashboard
- Download generated DOCX letter
- View all letters with status indicators and signatory completion

Technical notes: letter_docx.py uses python-docx>=1.1.0. Signatures stored as base64 PNG data-URLs in web/data/letters/. DOCX output at web/data/letters/docx/. Tracker events logged per letter.

1	Author	Compose letter	Subject, body, recipient, category
2	Project Manager	Apply PM signature	Base64 PNG, timestamp
3	Finance Manager	Apply FM signature	Base64 PNG, timestamp
4	Managing Director	Apply MD signature	Base64 PNG, timestamp
5	Any authorized	Mark as sent	Sent timestamp, tracker event

◆ 13. SIEM — Security Information & Event Management

SIEM Security Module

Built-in security monitoring and incident detection system that logs all user activity, file imports, and suspicious events with automated alert rules.

- Real-time event log: IP address, username, module, endpoint, HTTP method
- File upload event tracking: filename, file size, import row counts
- Automated alert rules: brute-force detection, high-volume imports, anomalies
- Alert severity levels: low / medium / high / critical
- Alert acknowledgement workflow
- Event filtering: by IP, module, status, date range
- Dashboard: event count by module, alert counts by severity
- Event detail view with full request metadata
- Audit trail middleware — every POST/PUT/PATCH/DELETE auto-logged
- User agent and referer captured on every event
- Content-type tracking for file upload classification
- Export events to CSV for external SIEM integration

Technical notes: siem_data_store writes to PostgreSQL siem_events and siem_alerts tables. Audit trail middleware logs at the HTTP layer, independent of individual route logic.

◆ 14. Backup & Archive System

Backup & Archive

Automated and on-demand backup system that compresses and archives application data files to local storage or AWS S3.

- Nightly automated backup scheduled at 01:00 via APScheduler
- on-demand manual backup trigger from dashboard
- Compressed archive format (gzip) with file count and size tracking
- Compression ratio reporting
- Backup log table: archive name, path, original/compressed size, timestamp
- Triggered-by field: 'scheduler', 'manual', or username
- Label support — annotate backups with human-readable notes
- Backup list view with restore status indicators
- AWS S3 upload capability for off-site archiving
- Retention management — configurable days to keep

Technical notes: BackupEngine in backup_data_store.py. BackupScheduler wraps APScheduler. Archives stored under /opt/ethiopian-business/web/data/backups/ (local) and optionally pushed to S3 bucket configured via S3_BUCKET env var.

◆ 15. Version Control Module

Version Registry

Lightweight application version registry that tracks deployed versions with changelog and activation status directly in the database.

- Version registry with unique version strings (semver)
- Version label and description
- Active / inactive status per version
- Changelog text per version entry
- Created-at timestamp
- Dashboard shows current active version
- Version history list
- Admin-only version management (promote / deactivate)

Technical notes: version_registry table in PostgreSQL. Managed via version_data_store.py. Useful for tracking which code version is running in multi-server deployments.

◆ 16. Multi-Company Portal

Multi-Company Portal

Allows a single user account to manage multiple companies from one login, with a full per-company subscription and module licensing engine.

- Company registration with unique `company_id`
- Company selection switcher — switch context mid-session
- Per-company subscription: Starter / Professional / Enterprise tier
- Module licensing: enable or disable individual modules per company
- Subscription expiry date tracking
- Max users and max employees limits enforced per tier
- Tenant dashboard: see all companies you belong to
- Row-Level Security at PostgreSQL level — data never crosses company boundaries
- Company name displayed in navigation bar
- Audit log per company: who changed what and when

Technical notes: `TenantDataStore` in `tenant_data_store.py` manages all tenants. `SUBSCRIPTION_TIERS` dict defines module sets. `ALWAYS_ALLOWED_MODULES = {auth, static}` bypasses licensing check.

Starter	Free	3	25	Auth, CoA, Journal, VAT
Professional	ETB 2,500/mo	15	200	Starter + Income/Expense, Transactions, CPO, Payroll
Enterprise	ETB 7,500/mo	Unlimited	Unlimited	Professional + Inventory, Bid, SIEM, Backup, Version, Multi-company

◆ 17. Learning Management System (LMS)

Learning Management System

Enterprise-grade Learning Management System with HR integration, course management, certifications, quizzes, gamification, and learning paths. Supports SCORM 1.2/2004, xAPI, and native content types.

- Course catalog with category filtering and skill level badges
- Multiple content types: text, video, PDF, SCORM 1.2, SCORM 2004, xAPI, HTML5, quiz
- Learning paths — multi-course sequences with progress tracking
- Certificate generation with validity periods and expiry tracking
- Quiz engine — multiple choice, true/false, multi-select questions
- Gamification — points, badges, leaderboards per company
- Self-enrollment and admin-assigned mandatory courses
- Compliance tracking — flag courses as compliance-required
- Progress tracking — percentage complete, time spent, last access
- HR integration — training requirements linked to employee records
- Machinery certification integration — operator qualifications verified
- Admin dashboard — course analytics, enrollment stats, completion rates
- Certificate download as PDF with QR verification code
- Expiring certificates alert — dashboard widget for upcoming renewals

Technical notes: lms_data_store.py manages 7 PostgreSQL tables: lms_courses, lms_learning_paths, lms_enrollments, lms_certificates, lms_quiz_attempts, lms_gamification, lms_badges. Content URLs support S3, external URLs, or local storage. SCORM packages tracked via scorm_data field.

/lms/	GET	LMS dashboard with stats and in-progress courses
/lms/courses	GET	Course catalog with filters
/lms/courses/{id}	GET	Course detail and syllabus
/lms/courses/{id}/enroll	POST	Self-enroll in a course
/lms/courses/{id}/learn	GET	Course learning interface
/lms/courses/{id}/complete	POST	Mark course as completed
/lms/my-learning	GET	User's enrolled courses and progress
/lms/certificates	GET	User's earned certificates
/lms/certificates/{id}/download	GET	Download certificate PDF
/lms/quiz/{id}	GET	Take quiz interface
/lms/quiz/{id}/submit	POST	Submit quiz answers
/lms/paths	GET	Learning paths catalog
/lms/leaderboard	GET	Gamification leaderboard
/lms/badges	GET	Available and earned badges
/lms/admin	GET	Admin dashboard (admin only)

/lms/admin/courses	GET	Manage courses (admin only)
/lms/admin/courses/add	GET/POST	Create new course

LMS Database Tables

lms_courses	Course definitions and metadata	course_id, title, content_type, duration_minutes, passing_score
lms_learning_paths	Multi-course sequences	path_id, course_ids (JSONB), grants_certification
lms_enrollments	User course enrollments	enrollment_id, user_id, course_id, status, progress_percent
lms_certificates	Issued certificates	certificate_id, user_id, course_id, issued_at, expires_at
lms_quiz_attempts	Quiz submission history	attempt_id, quiz_id, user_id, score, answers (JSONB)
lms_gamification	User points and levels	user_id, total_points, current_level, badges (JSONB)
lms_badges	Badge definitions	badge_id, name, criteria, icon_url, points_value

◆ 18. Machinery & Equipment Management

Machinery & Equipment

Full lifecycle construction equipment management for heavy machinery, vehicles, and tools. Includes asset tracking, site transfers, maintenance scheduling, operator shift logs, fuel logging, and HR/LMS certification integration.

- Asset registry — serial number, VIN, QR code, barcode tracking
- Asset categories: excavators, loaders, trucks, cranes, generators, tools
- Status tracking: available, in_use, maintenance, repair, decommissioned
- Ownership types: owned, leased, rented, subcontracted
- Site management — project sites, yards, workshops with geolocation
- Transfer orders — scheduled moves with approval workflow (pending → approved → in_transit → completed)
- Maintenance scheduling — preventive, corrective, inspection work orders
- Service interval tracking — hours-based or date-based triggers
- Operator shift logs — start/end times, hour meter readings, work performed
- Fuel log tracking — quantity, cost, consumption rate analysis
- Operator assignment — with LMS certification verification
- HR integration — required licenses and training courses per asset type
- Utilization reports — hours worked, idle time, underutilized alerts
- Project cost allocation — equipment costs rolled up by project/site
- API endpoints for mobile integration and GPS trackers

Technical notes: machinery_data_store.py manages 6 PostgreSQL tables. Assets linked to HR employees via operator_id. Certification verification calls LMS to confirm valid training. technical_specs, utilization, and financial stored as JSONB for flexible schema.

/machinery/	GET	Dashboard with fleet status and alerts
/machinery/assets	GET	Asset registry with filters
/machinery/assets/new	GET/POST	Add new asset form and handler
/machinery/assets/{id}	GET	Asset detail with history tabs
/machinery/assets/{id}/edit	GET/POST	Edit asset details
/machinery/sites	GET	Sites and yards list
/machinery/sites/new	GET/POST	Add new site/yard
/machinery/sites/{id}	GET	Site detail with assigned assets
/machinery/transfers	GET	Transfer orders list
/machinery/transfers/new	GET/POST	Create transfer order
/machinery/transfers/{id}	GET	Transfer detail and status
/machinery/transfers/{id}/approve	POST	Approve pending transfer
/machinery/maintenance	GET	Maintenance work orders
/machinery/maintenance/new	GET/POST	Create maintenance order

/machinery/maintenance/{id}/complete	POST	Complete maintenance with actuals
/machinery/shift-logs	GET	Operator shift logs
/machinery/shift-logs/start	GET/POST	Start operator shift
/machinery/shift-logs/{id}/end	POST	End shift with readings
/machinery/fuel-logs	GET	Fuel logs by asset
/machinery/fuel-logs/new	GET/POST	Log fuel purchase
/machinery/reports/utilization	GET	Fleet utilization report
/machinery/reports/project-cost	GET	Project equipment cost report

Machinery Database Tables

machinery_assets	Equipment/vehicle registry	asset_id, serial_number, category, status, current_site_id
machinery_sites	Sites, yards, workshops	site_id, name, site_type, latitude, longitude, geofence_radius_m
machinery_transfers	Site-to-site transfer orders	transfer_id, asset_id, from_site_id, to_site_id, status
machinery_maintenance	Maintenance work orders	work_order_id, asset_id, maintenance_type, priority, status
machinery_shift_logs	Operator shift records	log_id, asset_id, operator_id, start_time, end_time, hours_worked
machinery_fuel_logs	Fuel purchase records	log_id, asset_id, quantity_liters, cost_per_liter, hour_meter_reading

Asset Category Enums (models/machinery.py)

AssetCategory: excavator, loader, dozer, grader, roller, crane, forklift, truck, pickup, tanker, generator, compressor, welder, concrete_mixer, pump, scaffolding, hand_tools, survey_equipment, safety_equipment, office_trailer, other. AssetType provides further specificity (e.g., excavator_crawler, loader_backhoe). FuelType: diesel, petrol, electric, hybrid. MaintenanceType: preventive, corrective, inspection, emergency.

◆ 19. REST API v1 & v2

Two API versions available: **/api/v1/** (web dashboard integration) and **/api/v2/** (mobile app optimized). Both accept and return JSON. Authentication uses Bearer tokens issued from the portal. Interactive documentation at **/api/docs** (Swagger UI) and **/api/redoc** (ReDoc).

/api/v1/health	GET	Public	Service health check — returns {status, service}
/api/v1/whoami	GET	User	Current user info and company context
/api/v1/employees	GET	User	List all employees for current company
/api/v1/employees/{id}	GET	User	Single employee detail
/api/v1/employees	POST	Admin	Create new employee
/api/v1/siem/events	GET	Admin	List recent SIEM security events
/api/v1/dashboard/stats	GET	User	Aggregate counts: accounts, transactions
/api/v1/export/{module}	GET	User	Export module data to CSV or XLSX
/api/v2/sync/employees	GET	User	Mobile-optimized employee sync
/api/v2/sync/inventory	GET	User	Mobile inventory sync with delta support
/machinery/api/assets	GET	User	Machinery assets JSON endpoint
/machinery/api/check-certification	GET	User	Verify operator certification for asset

Export Endpoint — Supported Modules

employees	employee_data_store	CSV, XLSX
cpo	cpo_data_store	CSV, XLSX
transactions	transaction_data_store	CSV, XLSX
vat_income	vat_data_store → get_all_income	CSV, XLSX
vat_expenses	vat_data_store → get_all_expenses	CSV, XLSX
inventory	inventory_data_store	CSV, XLSX
lms_courses	lms_data_store	CSV, XLSX
machinery	machinery_data_store	CSV, XLSX

Sensitive fields (password, password_hash, secret) are automatically stripped from all export files.

◆ 20. Subscription Tiers & Licensing

EBMS uses a per-company subscription model enforced at both the application and database layers. Each tier unlocks a defined set of modules.

Starter Tier — Free

Core accounting package for small businesses and startups.

Max users: **3** · Max employees: **25**

Included modules: Auth, Chart of Accounts, Journal Entries, VAT Portal

Professional Tier — ETB 2,500 / month

Full financial operations suite for growing businesses.

Max users: **15** · Max employees: **200**

Included modules: Starter + Income & Expense, Transactions, CPO, Payroll

Enterprise Tier — ETB 7,500 / month

Complete business platform for large organisations.

Max users: **Unlimited** · Max employees: **Unlimited**

Included modules: Professional + Inventory, Bid Tracker, SIEM, Backup, Version Control, Multi-company

◆ 21. Security & Compliance

EBMS is designed to meet OWASP Top-10 requirements and Ethiopian tax compliance standards. The following controls are implemented at the framework and database levels:

Broken Access Control

Role hierarchy (super_admin > admin > manager > user). Module license gate enforced per tenant at middleware level. Row-Level Security in PostgreSQL prevents cross-tenant data access even if application WHERE clause is omitted.

Cryptographic Failures

Passwords hashed with bcrypt (cost factor 12). API token secrets stored as SHA-256 hash. Session secret key: 256-bit random hex from os.environ. HTTPS enforced by ALB; SESSION_COOKIE_SECURE=true sets Secure cookie flag.

Injection (SQL)

All database queries use parameterised statements via psycopg2 cursor.execute(). No string concatenation into SQL. ORM pattern via db.py helper.

Injection (XSS)

Jinja2 auto-escaping on all templates. User input rendered with {{ var }} (escaped). Content-Security-Policy header set on static assets.

CSRF

Dual CSRF strategy: hidden csrf_token field injected into every HTML POST form by middleware; X-CSRFToken header validated on all AJAX/HTMX mutations.

Rate Limiting

SlowAPI (slowapi) middleware enforces per-endpoint rate limits. Login endpoint limited to prevent brute-force attacks.

Audit Trail

Every POST/PUT/PATCH/DELETE request is logged to the SIEM events table with username, IP, path, and timestamp via audit_trail_middleware.

Session Security

Sessions stored server-side in Redis (fallback to signed cookie). SameSite=Lax prevents CSRF via cross-site navigation. Server-side idle expiry after 30 minutes of inactivity (configurable via SESSION_IDLE_MAX_MINUTES) — enforced in middleware, not just client-side.

Dependency Security

requirements.txt is fully pinned to exact versions (pip freeze); direct dependencies documented in requirements.in. CI pipeline (Azure Pipelines) runs the test suite against live PostgreSQL 16 and Redis 7 containers on every push. Secrets live in an untracked .env file — never in code or git.

Ethiopian Tax Compliance

VAT calculations aligned with ERCA proclamation. Payroll PAYE follows Ethiopian Income Tax Proclamation No. 979/2016. Pension rates: employee 7%, employer 11% per labour proclamation.

◆ 22. Deployment & Infrastructure

EBMS runs on-premises on a dedicated server (Dell PowerEdge R430, Ubuntu) as a six-service **Docker Compose** stack. The application is reachable at <https://ebms.devopsrain.com> with a Let's Encrypt certificate; the server is **not exposed to the public internet** — all remote access goes through a Tailscale (WireGuard) VPN. Deployment is git-driven: push to Azure DevOps, pull on the server, rebuild containers.

Host	Dell PowerEdge R430	Ubuntu, Docker Engine + Compose, user devopsrain
Reverse proxy	Nginx 1.27 (container)	TLS termination :443, proxies web :8000 and api :8001
TLS certificate	Let's Encrypt	certbot DNS-01 via AWS Route 53; auto-renew (systemd timer)
Application	web / api / event-worker	Uvicorn ASGI containers, 2 workers, health-checked
Database	PostgreSQL 16 (container)	Data on named Docker volume, port bound to 127.0.0.1
Cache / events	Redis 7 (container)	Sessions + pub/sub event bus, appendonly persistence
Remote access	Tailscale VPN	Subnet routing 192.168.68.0/24 — no port forwarding
Backups	cron + deploy/backup.sh	Nightly pg_dump + app-data archive, 30-day retention
CI	Azure Pipelines	Tests against live Postgres 16 + Redis 7 on every push
Deploy workflow	git pull + compose build	No manual file edits on the server; config via .env/override

Environment Variables

FLASK_SECRET_KEY	Yes	256-bit session signing key
DATABASE_URL	Yes	postgresql://user:pass@host:5432/db
POSTGRES_PASSWORD	Yes	Database password (server-only .env, never in git)
REDIS_URL	Optional	redis://host:6379/0 — session and cache backend
SESSION_COOKIE_SECURE	Prod	'1' enforces HTTPS-only session cookie
SESSION_IDLE_MAX_MINUTES	Optional	Server-side idle logout threshold (default: 30)
LOG_LEVEL	Optional	DEBUG / INFO / WARNING / ERROR (default: INFO)
S3_BUCKET	Optional	S3 bucket name for off-site backup uploads
PROVIDER_ADMIN_PASSWORD	Optional	Provider admin portal password — portal disabled (fail-closed) when unset

◆ 23. Database Schema Reference

Complete list of PostgreSQL tables used by EBMS modules. All tables support multi-tenancy via `company_id` column and Row-Level Security policies. The full schema is created idempotently at application startup from `aws-deployment/init_db.sql` plus per-module `ensure_schema()` initializers run in the FastAPI lifespan; Alembic migrations apply incremental ALTERs on upgrade.

Core Business Tables

accounts	Chart of Accounts	account_id	Account codes and hierarchy
journal_entries	Journal Entry	entry_id	Double-entry journal headers
journal_lines	Journal Entry	line_id	Debit/credit lines per entry
employees	Payroll	employee_id	Employee master data
payroll_runs	Payroll	run_id	Monthly payroll calculations
vat_income	VAT/Income-Expense	record_id	Income transactions with VAT
vat_expenses	VAT/Income-Expense	record_id	Expense transactions with VAT
vat_capital	VAT Portal	capital_id	Capital investments
transactions	Transactions	transaction_id	Bank statement imports
flagged_accounts	Transactions	account_number	Auto-flag account registry

Inventory & Equipment Tables

inventory_items	Inventory	item_id	Stock items with valuation
inventory_movements	Inventory	movement_id	Stock movements (in/out/transfer)
inventory_requisitions	Inventory	requisition_id	Stock requests
inventory_allocations	Inventory	allocation_id	Event/project allocations
machinery_assets	Machinery	asset_id	Heavy equipment registry
machinery_sites	Machinery	site_id	Project sites and yards
machinery_transfers	Machinery	transfer_id	Site-to-site transfer orders
machinery_maintenance	Machinery	work_order_id	Maintenance work orders
machinery_shift_logs	Machinery	log_id	Operator shift records
machinery_fuel_logs	Machinery	log_id	Fuel purchase records

LMS & HR Tables

lms_courses	LMS	course_id	Course definitions and content
lms_learning_paths	LMS	path_id	Multi-course learning paths
lms_enrollments	LMS	enrollment_id	User course enrollments
lms_certificates	LMS	certificate_id	Issued certificates

lms_quiz_attempts	LMS	attempt_id	Quiz submission history
lms_gamification	LMS	user_id	Points and level tracking
lms_badges	LMS	badge_id	Badge definitions

Security & System Tables

users	Auth	user_id	User accounts and credentials
api_tokens	Auth	token_id	Bearer API tokens
companies	Multi-Company	company_id	Tenant definitions
user_companies	Multi-Company	id	User-to-company associations
siem_events	SIEM	event_id	Security event log
siem_alerts	SIEM	alert_id	Security alerts
backup_log	Backup	backup_id	Backup history
version_registry	Version	version_id	Deployed versions
letters	Letters	letter_id	Letter metadata (JSON file-backed)
cpo_records	CPO	cpo_id	Cash payment orders
bid_records	Bid Tracker	bid_id	Tender/bid tracking
bid_documents_meta	Bid Tracker	document_id	Bid document attachments

Data Store Files (Technical Reference)

employee_data_store.py	employees, payroll_runs	add_employee, calculate_payroll, get_payslip
inventory_data_store.py	inventory_items, _movements, _requisitions, _allocations	add_item, record_movement, get_valuation
lms_data_store.py	lms_courses, _enrollments, _certificates, _gamification	enroll_user, complete_course, issue_certificate
machinery_data_store.py	machinery_assets, _sites, _transfers, _maintenance, _shift_logs	create_asset, start_shift, complete_maintenance
transaction_data_store.py	transactions, flagged_accounts, import_history	bulk_import, flag_account, get_import_batch
vat_data_store.py	vat_income, vat_expenses, vat_capital	add_income, add_expense, get_financial_summary
auth_data_store.py	users, api_tokens	authenticate, create_user, generate_token
siem_data_store.py	siem_events, siem_alerts	log_upload_event, create_alert, get_dashboard_stats

◆ 24. Operations Suite Modules

Beyond the core financial modules described in sections 3–18, EBMS ships an Operations Suite of eight additional modules, each registered as an independent router and covered by the same authentication, licensing, audit, and multi-tenancy middleware stack.

Human Resources (HRM)	/hrm/	Leave management, attendance, performance reviews, recruitment — integrated with payroll and LMS
Finance Management	/finance/	Budgeting, cash-flow oversight, and consolidated financial KPIs across all accounting modules
Project Management	/project/	Projects, tasks, milestones, team assignments, budget tracking, and per-project cost roll-ups
Procurement	/procurement/	Purchase requisitions, supplier management, and approval workflows linked to inventory and CPO
Event Management (EMS)	/ems/	Venue and event planning with equipment allocation from inventory and per-event cost tracking
Communication Platform	/communication/	Internal channels and messaging for team coordination inside EBMS
Forecasting & Analytics	/forecast/	Predictive analytics over revenue, expenses, and inventory trends
Seamless UX	/search/ · /notifications/	Global search, in-app notifications, and health endpoints shared across every module

Module schemas are created at startup by their `ensure_schema()` initializers (`project_store`, `comm_store`, `ems_store`, `procurement_store`, `notifications_store`) invoked from the FastAPI lifespan.

◆ 25. Ethiopian-Native, Open Platform & Workflow (v2.2)

Release 2.2 makes EBMS a genuinely Ethiopian product (Amharic interface, Ethiopian calendar, mobile-money and ERCA outputs), opens it to customers, suppliers and other systems (portals, webhooks, API keys, Nextcloud, Telegram), and deepens the workflow layer (configurable approvals, fixed assets, a self-service report builder).

25.1 Ethiopian-Native

Amharic Localization & Ethiopian Calendar

Bilingual interface (English / ) with a one-click language switcher and the Ethiopian calendar () shown alongside every Gregorian date.

- Per-user language preference (session + cookie); Amharic catalogue of 1,000+ UI strings
- Noto Sans Ethiopic loaded automatically when Amharic is active
- Today's Ethiopian date in the navigation bar; `|et_date` and `|dual_date` template filters
- E.C. badge and picker under every date input — users type Ethiopian dates, the system stores ISO
- Ethiopian fiscal-year helpers (Hamle 1 – Sene 30), Ge'ez numerals for formal documents
- Exact Julian-Day conversion (Pagume 5/6, leap years) with unit tests against known dates

Technical notes: `web/i18n.py`, `web/ethiopian_calendar.py`, `static/js/ethiopian-calendar.js`. No gettext toolchain: `_('text')` falls back to English when a key is missing.

Mobile Money Payments

Records and reconciles Telebirr, CBE Birr, M-Pesa, bank and cash receipts and payments.

- Payment accounts per provider with balances and daily/monthly in/out summaries
- Manual entry, Excel/CSV statement import (per-provider templates, duplicate-safe on transaction ID)
- Provider notifications received on `/webhooks/inbound/` and parsed by provider adapters
- Reconciliation workbench: suggested matches against income/expense records scored by amount, date and reference
- MSISDN normalisation (+2519..., 09..., 07...), reversals, raw-payload viewer
- Provider credentials optional — record-only mode until Telebirr / Safaricom / CBE onboarding is complete

Technical notes: `payment_providers.py` adapters; `payments_data_store.py`; `PAYMENT_PROVIDERS.md` documents the env variables.

ERCA Tax Outputs & E-Invoice Numbering

Ministry of Revenues-ready declarations generated from the data already in the VAT portal.

- Monthly VAT return computed from VAT income/expenses (output, input, credit carried forward, net)
- Withholding register with 2% / 30% rules, monthly withholding return, withholding receipts
- Gapless, concurrency-safe invoice/receipt series with voided-number audit — numbers are never reused
- Tamper-evident SHA-256 hash chain across issued invoices
- Taxpayer profile (TIN, VAT registration, tax centre, Amharic/English names)
- PDF and Excel exports; audit export (JSON + Excel) for ERCA inspections

Technical notes: erca_forms.py (pure computations), erca_data_store.py, reportlab PDFs. Rates are constants marked 'verify against current proclamation'.

25.2 Open Platform

Customer & Supplier Portal

Invite-only self-service portal so customers and suppliers stop e-mailing for status updates.

- Separate portal identity, CSRF protection, login rate limiting and account lockout
- Customers: invoices/income records, CPOs, projects, support tickets with threaded replies
- Suppliers: purchase orders, payments, RFQ invitations and responses, document upload (quotes, invoices, delivery notes)
- Strict party scoping — a portal user only ever sees rows matching their own name/TIN and company
- Staff admin: user management, invitations, ticket inbox, RFQ management, document review
- Mobile-first public layout with the tenant's company name

Technical notes: portal_auth.py, portal_data_store.py; invite/reset tokens expire (72 h / 1 h); Cache-Control: no-store.

Webhooks & Per-Tenant API Keys

Lets other systems subscribe to EBMS events and call the API with scoped, revocable keys.

- Outbound webhooks with HMAC-SHA256 signatures, event wildcards (payment.*), retries with exponential backoff
- SSRF guard refuses private/loopback targets; delivery log with request/response and manual retry
- API keys hashed at rest, shown once, scoped, expiring, revocable; drop-in FastAPI dependency
- Signed inbound receiver for provider callbacks; developer docs page with Python and Node examples
- Event catalogue: invoice.*, payment.*, bid.*, purchase_requisition.approved, approval.decided, employee.created, asset.disposed

Technical notes: webhook_data_store.emit(company_id, event, payload); api_keys.require_api_key.

Documents on Nextcloud & Telegram Bot

Files live in the company's own Nextcloud; day-to-day figures arrive on Telegram.

- WebDAV document backend with local-disk fallback, folder structure per company/module/entity
- Browse, search, versioning, expiring share links (OCS), sync of files added directly in Nextcloud
- Telegram: link a chat with a one-time code; /today /month /bids /approvals /stock /payments
- Approve or reject requests from inline buttons; 07:30 daily digest; topic subscriptions
- Bilingual bot replies (■■■ / Income); webhook secret enforced; outbox with retry

Technical notes: nextcloud_client.py, document_storage.py facade; telegram_bot.py with a pure command dispatcher.

25.3 Workflow Depth

Configurable Approval Engine

Amount-banded, multi-step approvals that any module can plug into.

- Workflows per entity type (purchase requisition, leave, expense, payment, generic) with min/max amount bands
- Sequential steps; approver by role, named user or requester's manager; require-all steps
- Delegation windows, comments, cancel, escalation reminders for overdue requests
- Inbox for approvers, timeline per request, company-wide history
- Completion hooks update the source record (PR approved/rejected, leave decided) and emit approval.decided webhooks
- Already wired: purchase requisition submission and leave requests

Technical notes: `approval_store.submit()` returns `None` when no workflow matches so callers keep their manual path.

Fixed Assets & Depreciation

Asset register with Ethiopian tax-class defaults and automatic monthly depreciation.

- Categories seeded per company: buildings 5% SL, intangibles 10% SL, computers 25%, other assets 20%
- Straight-line, declining balance (switching), sum-of-years-digits, units-of-production schedules
- Idempotent monthly run (day 1, 02:00) with preview and confirm; never below salvage value
- Posting to the general ledger (depreciation expense / accumulated depreciation)
- Disposals with gain/loss, maintenance log, asset register Excel export, dashboard charts

Technical notes: Decimal arithmetic quantised to 2 dp; final period plugs rounding exactly to salvage.

Report Builder, PDF Export & Scheduled Reports

Self-service reporting over a whitelisted catalogue of 28 data sources — no SQL, no developer.

- Pick source, columns, filters, grouping, aggregates, sort, date presets (incl. Ethiopian fiscal year)
- Preview, save, share; bar/line/pie charts; HTML, Excel, CSV and PDF output
- Prebuilt templates: income by month, expenses by category, VAT summary, bids by status, payroll cost, stock valuation
- Daily / weekly / monthly schedules e-mailed with attachments; run history with downloads
- Safe query compiler: only catalogue identifiers, parameterised values, company scoping always enforced

Technical notes: `reports_catalog.py`, `reports_engine.py`, `reports_jobs.py` (15-minute scheduler tick).

◆ Appendix A — Application Screenshots

The following screenshots show the EBMS user interface across key modules. Each screen mirrors the live application layout including sidebar navigation, KPI cards, data tables, and status badges.

Figure 1 — Main Dashboard

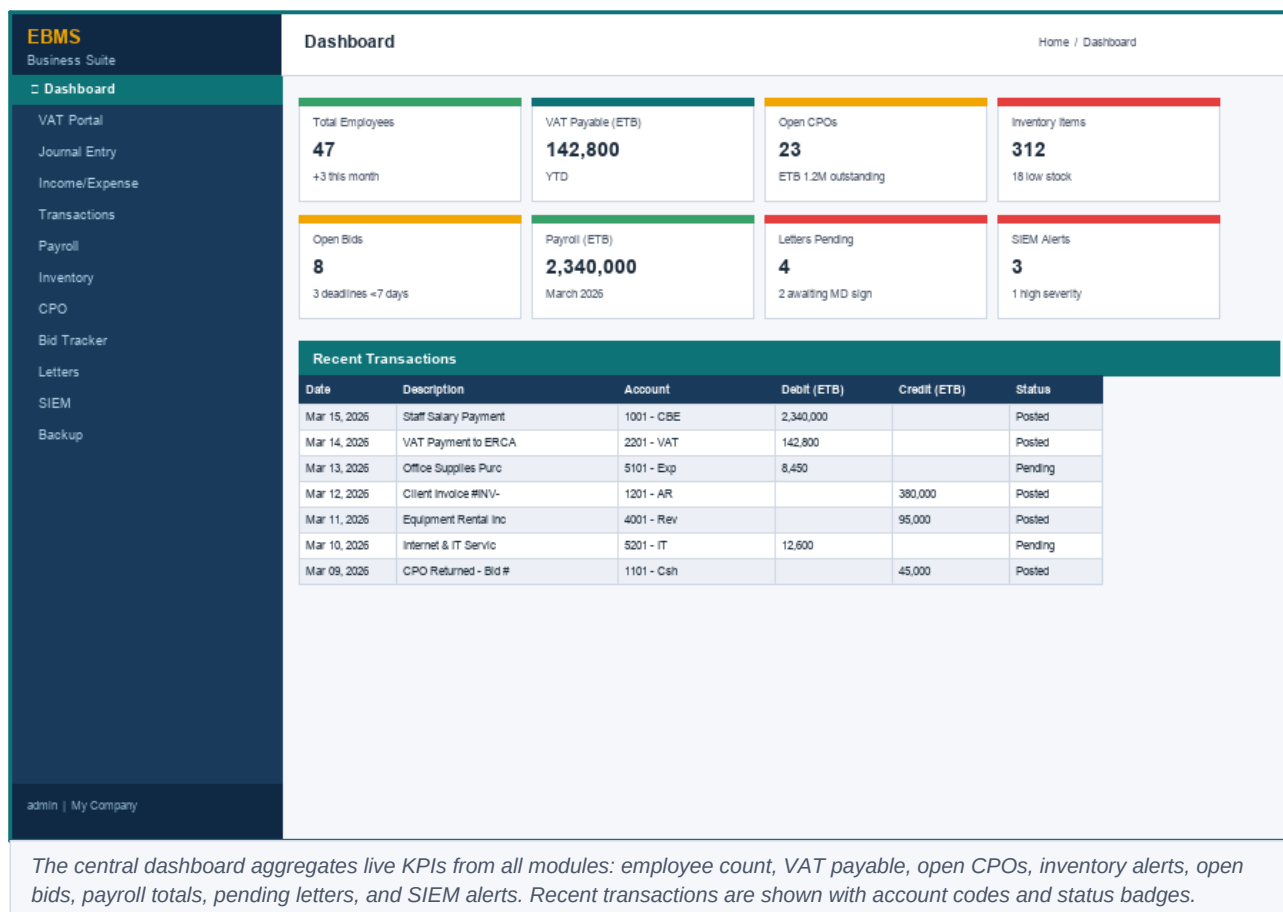


Figure 2 — Payroll Module — Employee List

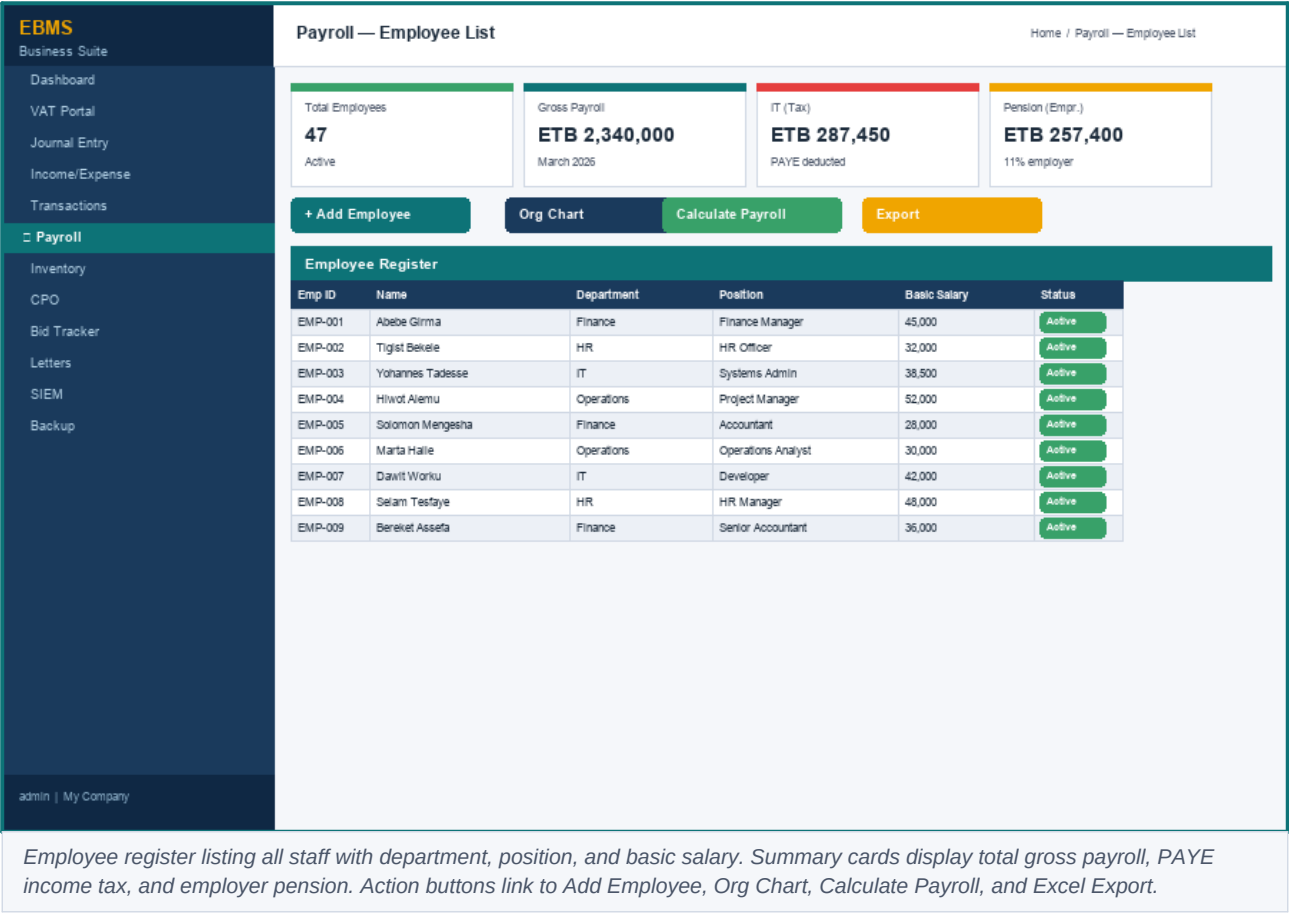


Figure 3 — VAT Portal — Financial Summary

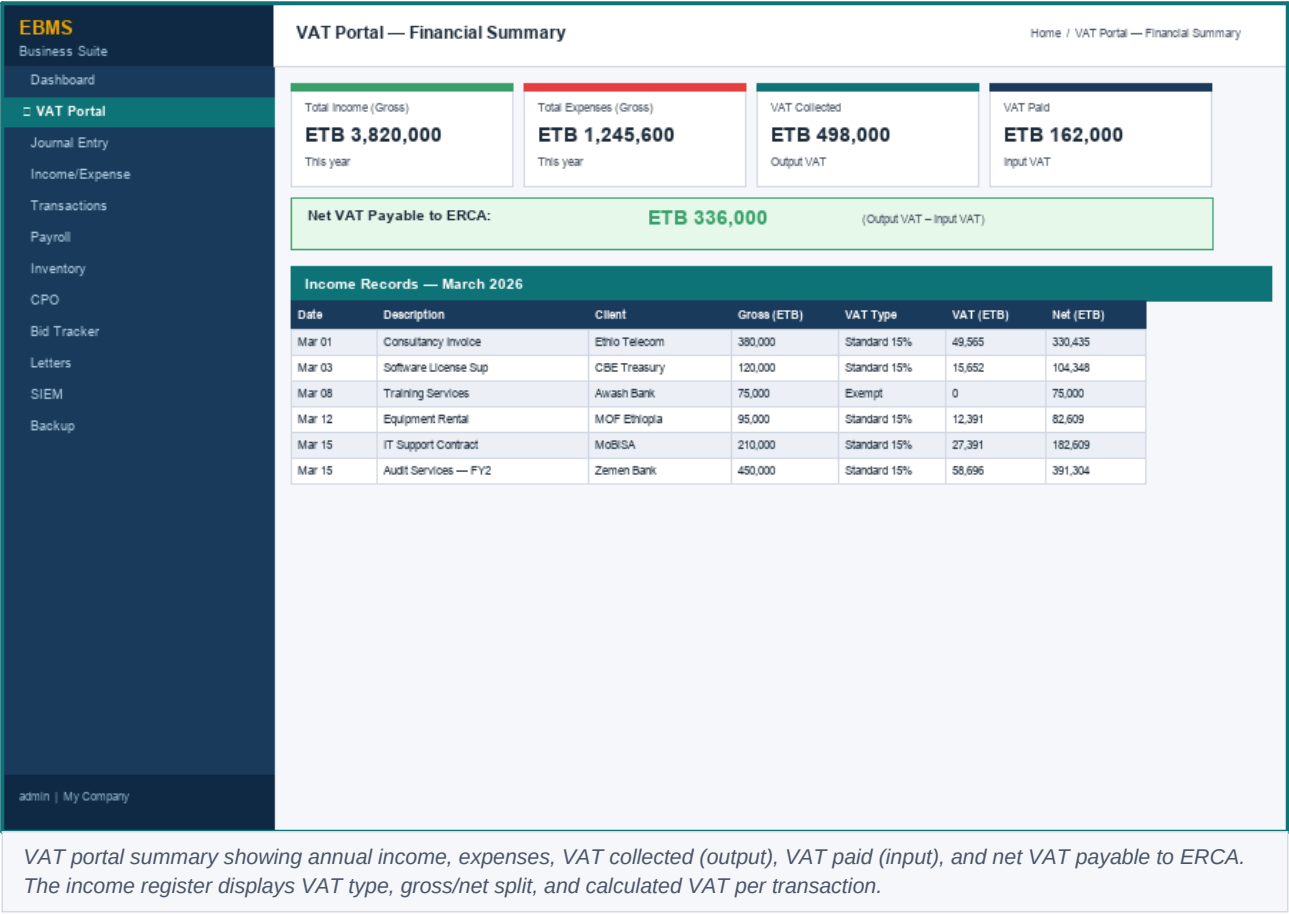
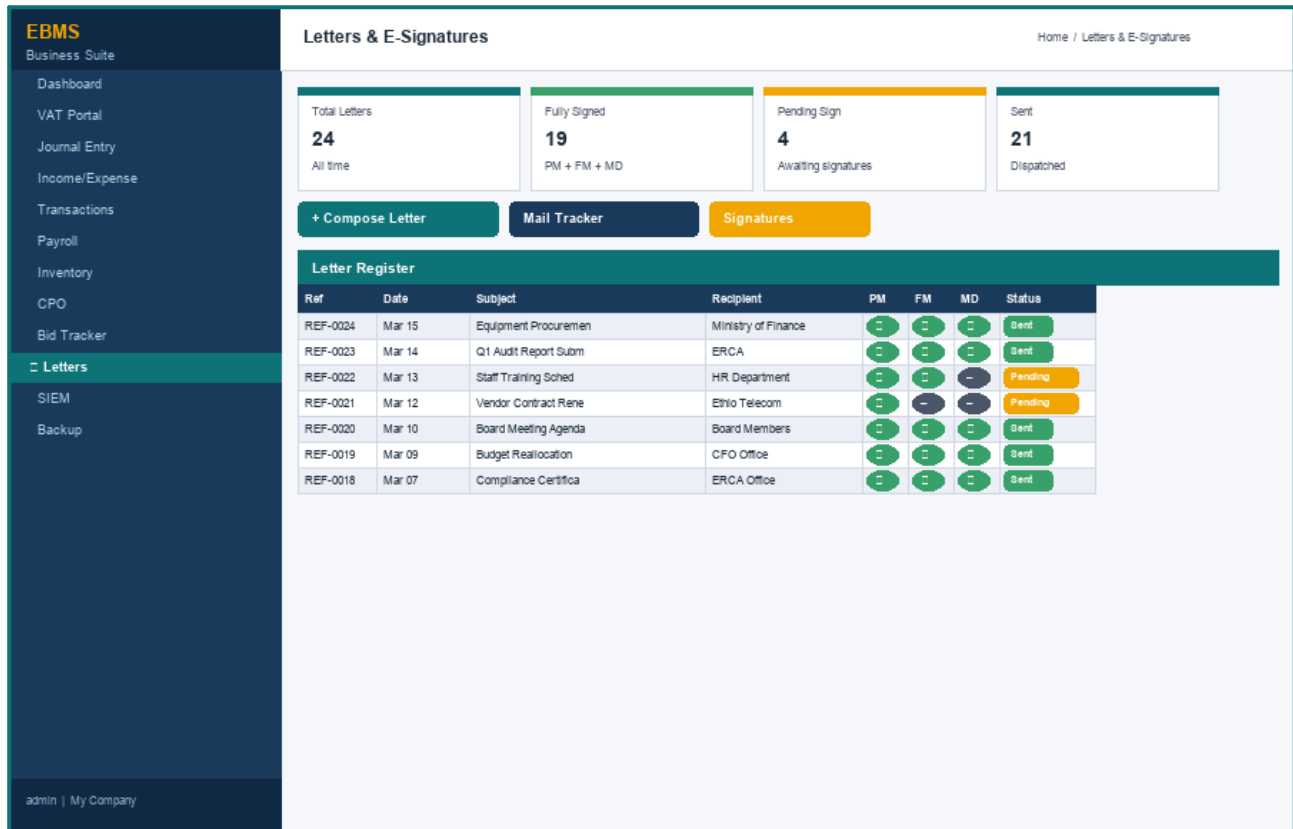


Figure 4 — Letters & E-Signatures



Letter register with sequential REF-xxxx references, PM/FM/MD signature status indicators (green = signed, grey = pending), and status badges. Navigation links to Compose, Mail Tracker, and Signature drawing pads.

Figure 5 — Inventory Management

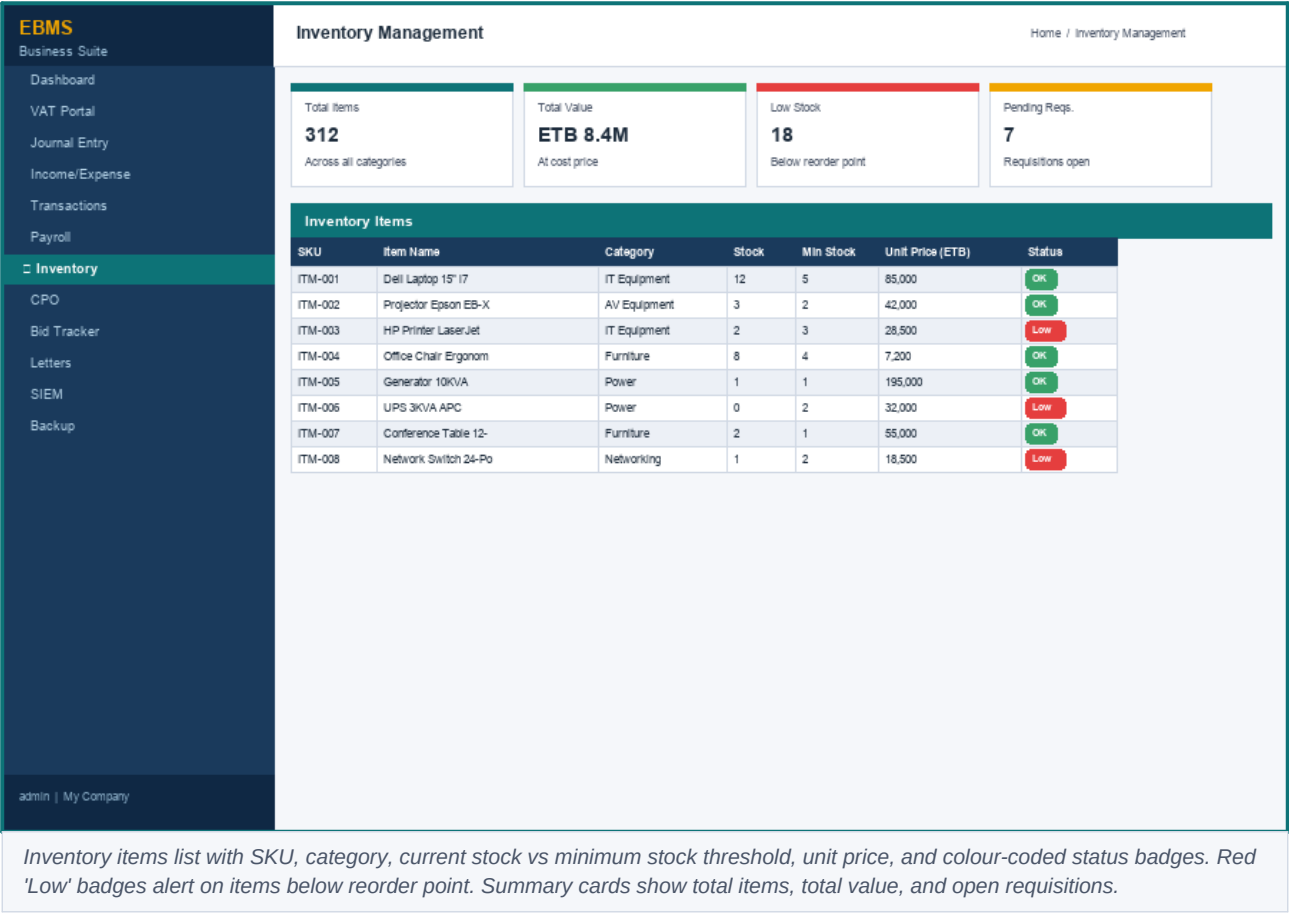
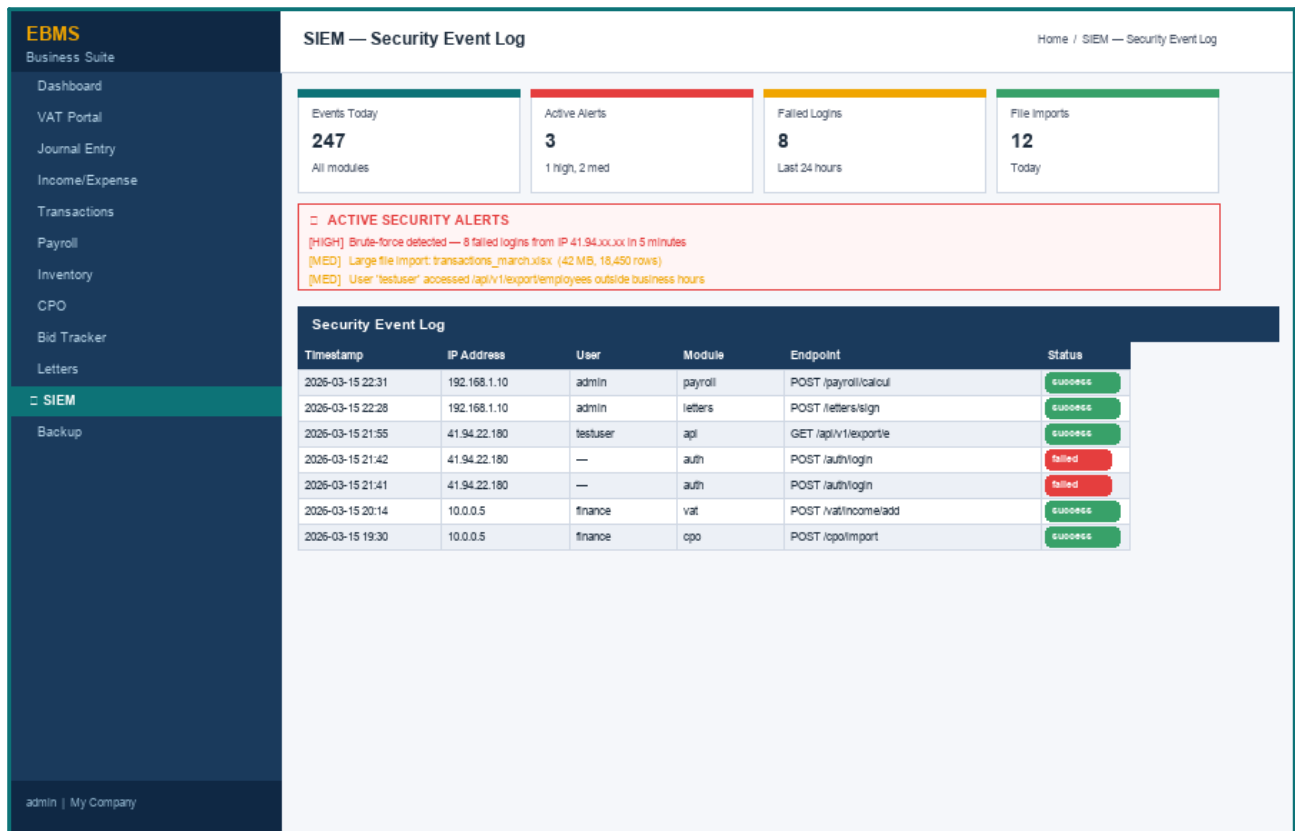
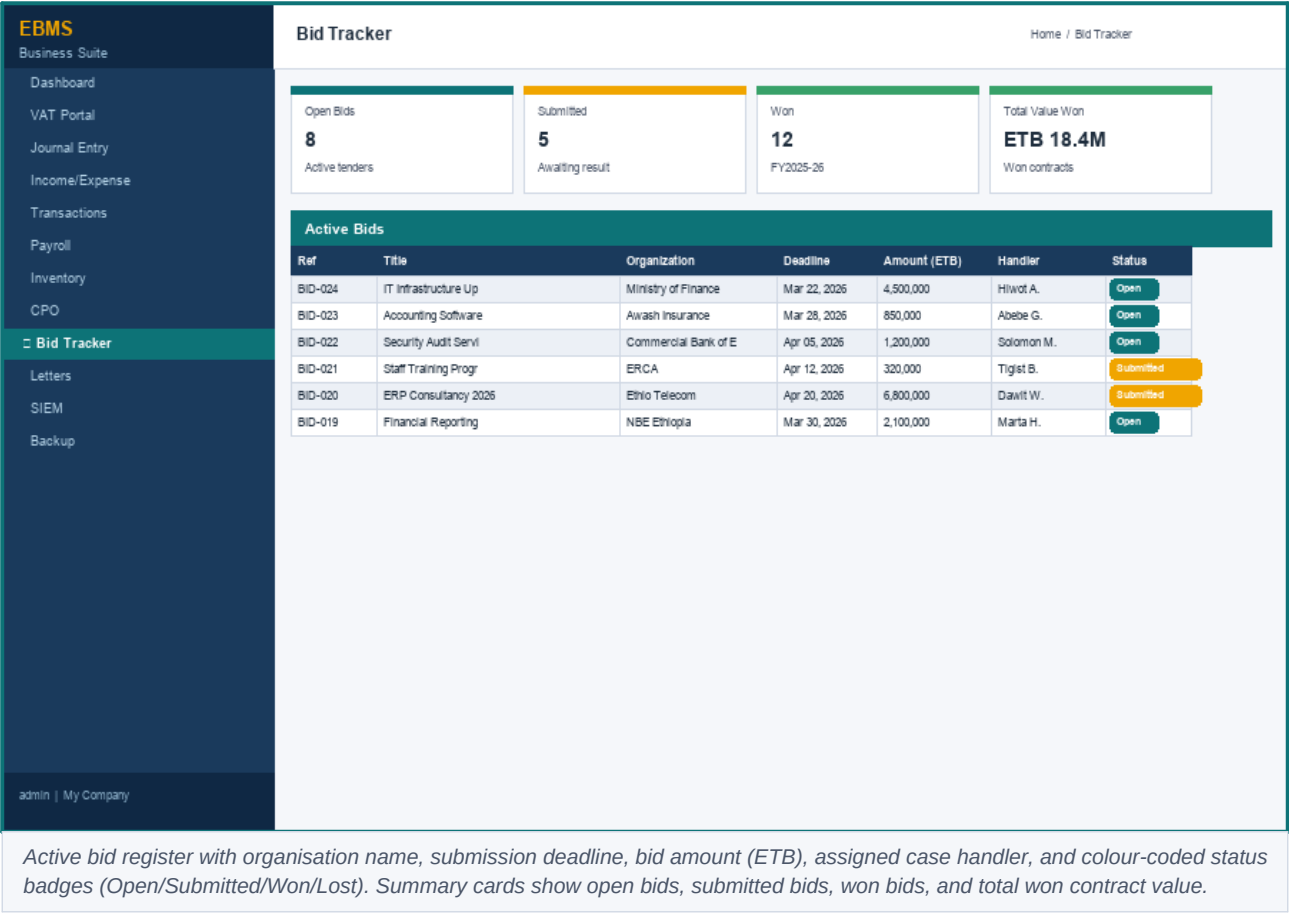


Figure 6 — SIEM Security Event Log



Security dashboard showing active HIGH/MEDIUM alerts in a red alert panel, followed by the full event log with IP addresses, authenticated usernames, module paths, endpoint URLs, and success/failure status badges for every HTTP mutation.

Figure 7 — Bid Tracker



◆ Appendix B — Sample Data Sheet

Representative sample records from each major module, illustrating the data fields, value formats, and Ethiopian-specific content stored in EBMS.

B1. Employee Register (Sample)

EMP-001	Abebe Girma	Finance	Finance Manager	ETB 45,000	2021-03-01	0021384
EMP-002	Tigist Bekele	HR	HR Officer	ETB 32,000	2022-06-15	0038271
EMP-003	Yohannes Tadesse	IT	Systems Admin	ETB 38,500	2020-09-01	0049153
EMP-004	Hiwot Alemu	Operations	Project Manager	ETB 52,000	2019-01-10	0061847
EMP-005	Solomon Mengesha	Finance	Accountant	ETB 28,000	2023-02-20	0074392

B2. Payroll Calculation — March 2026 (EMP-004: Hiwot Alemu)

Basic Salary	52,000	Gross monthly basic
Transport Allowance	2,000	Non-taxable per proclamation
Housing Allowance	3,000	Taxable allowance
Overtime (8 hrs x 1.25)	2,500	Regular overtime rate
Gross Taxable Income	57,500	Basic + taxable allowances + OT
Income Tax (PAYE)	-14,575	35% bracket: ETB 7,800 threshold
Employee Pension (7%)	-3,640	7% of basic salary
CPO Deduction	0	No outstanding CPO this period
NET PAY	41,285	Deposited to CBE account
Employer Pension (11%)	5,720	Additional cost to organisation

B3. VAT Income Records (Sample — March 2026)

2026-03-01	Consultancy Invoice #042	Ethio Telecom	380,000	Std 15%	49,565	330,435
2026-03-03	Software License Supply	CBE Treasury	120,000	Std 15%	15,652	104,348
2026-03-08	Training Services	Awash Bank	75,000	Exempt	0	75,000
2026-03-12	Equipment Rental	MOF Ethiopia	95,000	Std 15%	12,391	82,609
2026-03-15	IT Support Contract Q1	MoBiSA	210,000	Std 15%	27,391	182,609

B4. Letter Register with Signature Timestamps (Sample)

REF-0024	Mar 15	Equipment Procurement Approval	Ministry of Finance	15-Mar 21:00	15-Mar 21:30	15-Mar 22:00	Sent

REF-0023	Mar 14	Q1 Audit Report Submission	ERCA	14-Mar 16:00	14-Mar 16:45	14-Mar 17:30	Sent
REF-0022	Mar 13	Staff Training Schedule FY2026	HR Department	13-Mar 09:00	13-Mar 09:30	—	Pending
REF-0021	Mar 12	Vendor Contract Renewal Notice	Ethio Telecom	12-Mar 14:00	—	—	Pending

B5. CPO Records (Sample)

CPO-001	Abebe Girma	2026-01-10	45,000	BID-018	Yes	2026-03-05
CPO-002	Ethio Suppliers Ltd	2026-01-15	120,000	BID-019	No	—
CPO-003	Solomon Mengesha	2026-02-01	30,000	BID-020	Yes	2026-02-28
CPO-004	Global Tech PLC	2026-02-14	280,000	BID-021	No	—
CPO-005	Tigist Bekele	2026-03-01	22,500	BID-022	No	—

B6. Inventory Items (Sample)

ITM-001	Dell Laptop 15" i7	IT Equipment	12	5	85,000	FIFO	OK
ITM-002	Projector Epson EB-X51	AV Equipment	3	2	42,000	FIFO	OK
ITM-003	HP Printer LaserJet	IT Equipment	2	3	28,500	FIFO	Low
ITM-004	Office Chair Ergonomic	Furniture	8	4	7,200	FIFO	OK
ITM-005	Generator 10KVA	Power	1	1	195,000	FIFO	OK
ITM-006	UPS 3KVA APC	Power	0	2	32,000	FIFO	Low

B7. SIEM Security Events (Sample)

2026-03-15 22:31:04	192.168.1.10	admin	payroll	POST /payroll/calculate	success
2026-03-15 22:28:17	192.168.1.10	admin	letters	POST /letters/sign	success
2026-03-15 21:55:42	41.94.22.180	testuser	api	GET /api/v1/export/employees	success
2026-03-15 21:42:11	41.94.22.180	—	auth	POST /auth/login	failed
2026-03-15 21:41:55	41.94.22.180	—	auth	POST /auth/login	failed
2026-03-15 20:14:33	10.0.0.5	finance	vat	POST /vat/income/add	success